

Feature Specification — FitA11y Fitness App

Version 2.0 — Assistive Playback Companion *Sequenced by order of use · Technical methods · Training vs. Prompt Engineering*

Legend

Tag	Meaning
[PE]	Prompt Engineering
[SFT]	Supervised Fine-Tuning
[STAT]	Statistical / Rule-based (no model)
[LOCAL]	Runs on device, no API
[BANDIT]	Contextual Bandit (adaptive)
	New introduction beyond AudioToDes baseline
	Revised from v1 spec

Changelog — v1 → v2

This section documents every substantive change from the original BLVAI / FitA11y v1 spec.

Product thesis change

v1: FitA11y downloaded the YouTube video, processed it into a manifest, and served the workout as an AI-generated, AI-narrated session. The AI was the primary trainer.

v2: FitA11y is an **assistive playback companion**. The original YouTube video — with the creator's audio and visuals — is the primary training experience. FitA11y adds an accessibility sidecar layer: haptic/spatial cues, form correction, rep counting, safety monitoring, and Q&A. The YouTuber remains the trainer of record at all times.

Structural changes

- Phase 1 renamed from "Video Preprocessing Pipeline" to "Assistance Preparation Pipeline." Preprocessing no longer generates a full workout script; it generates an **assistance sidecar**

manifest that annotates the original video with accessibility metadata.

- Phase 3 renamed from "Live Session" to "Live Assisted Playback." The embedded YouTube IFrame player is now the central UI element, not a reference source for a re-generated session.
- All features that implied the AI replaces, regenerates, or overrides the creator's workout content have been revised or removed.

Creator-first policy added

New standing rule applied to all Phase 3 features. See Section: Creator-First Policy.

Audio coexistence rules added

New section defining how assistant speech, trainer audio, and haptic/spatial output interact. Replaces the assumption in v1 that the AI's voice was the dominant audio channel.

Adaptive pacing revised

v1 allowed the system to reduce rep targets and offer alternative workouts as canonical session outcomes. v2 constrains this: rep reduction becomes a user-facing **adaptation recommendation**, and the canonical workout (as defined by the creator's video) is never rewritten. Pacing interventions are delivery adaptations only.

Preprocessing scope change

v1: Full workout manifest with synthesized descriptions, rep counts, haptic profiles, and correction templates — effectively a shadow workout script.

v2: Assistance sidecar manifest containing YouTube metadata, exercise timeline anchors, trainer instruction timestamps, expected movement windows, form risk profiles, correction templates, haptic/spatial cue mappings, and silence windows. No shadow script.

Features removed or deprecated

- **F1.3 Multi-Level Description Generation** — deprecated as a preprocessing step. Description level is now a live-session accessibility setting applied to the assistant's supplementary cues only, not to synthesized narration that competes with the trainer.
- **F3.1–F3.2** revised: rep counting is now supplementary confirmation, not the primary rep feedback mechanism (the trainer still counts and cues).

Open questions / product decisions added

See final section.

Creator-First Policy

The following rules govern all Phase 3 behavior and take precedence over any individual feature description.

- 1. **Do not over-talk the trainer.** If the trainer is speaking, FitA11y must not produce simultaneous voiced output unless the user has enabled an override mode.
- 2. **Do not regenerate the workout.** FitA11y does not produce an independent script, rep target sequence, or exercise narration. All voiced content from FitA11y is supplementary and contextual.
- 3. **Assistance must be brief and timed.** Spoken interventions target natural gaps between trainer instructions. The preferred delivery is haptic-first, audio only when necessary.
- 4. **The video is the source of truth.** Exercise names, rep targets, rest periods, and modifications are taken from the creator's video. FitA11y never contradicts the trainer.
- 5. **User override is always available.** The user can always dismiss an intervention, mute the assistant for a set, or ask FitA11y to stay silent for the rest of a segment.

Audio Coexistence Rules NEW

This section defines how assistant output (speech, haptics, spatial audio) coexists with the original trainer audio.

Priority hierarchy

- 1. **Trainer audio** — always has priority. Never suppressed.
- 2. **Safety alerts** — only FitA11y output that may interrupt trainer speech (e.g., pain report, critical form risk).
- 3. **Form corrections** — delivered in trainer speech gaps, or haptic-only if no gap is available.
- 4. **Motivational cues** — lowest priority; haptic-only during trainer speech unless a long natural gap is detected.
- 5. **Q&A responses** — video paused before speaking, trainer audio resumes after.

Delivery modes

Mode	Description	When used
Haptic-only	Sleeve patterns, no speech	During trainer speech, high-frequency exercises

Mode	Description	When used
Spatial audio cue	Directional tone or short chime	Body-part-specific cues during trainer speech
Duck + speak	Trainer audio lowered to configurable %, assistant speaks	Medium-gap corrections
Pause + speak	Video fully paused before assistant speaks	Q&A, safety alerts, pacing interventions
Gap-timed speech	Assistant waits for detected silence from trainer	Default for corrections and motivation

User settings for audio coexistence

All settings live in the user profile and are configurable before each session:

- **Interruption level:** ☐ Never interrupt / ☐ Haptic-only during speech / ☐ Duck-and-speak allowed / ☐ Always interrupt
- **Assistant verbosity:** ☐ Haptic + short cue (≤5 words) / ☐ Standard (≤10 words) / ☐ Descriptive (≤20 words)
- **Pause before speaking:** ☐ Always pause video / ☐ Pause only for Q&A / ☐ Never pause
- **Correction frequency:** ☐ Every error / ☐ Every 3 errors / ☐ Set summary only

PHASE 0 — Account & Hardware Setup

Runs once per user. Prerequisites to everything else.

F0.1 — User Profile & BLV Preferences

What: Account creation. Captures screen-reader type, degree of vision loss, preferred TTS voice, earpiece / speaker preference, and audio coexistence settings (see Audio Coexistence Rules above). Stores accessible UI preferences (keyboard nav, no-animation mode, high-contrast toggle).

How: Standard Next.js auth (Supabase). The profile object — including audio coexistence settings and persona preference — is passed as context into every LLM system prompt throughout the app.

Method: ☐ [STAT] No model involved. Form → database.

F0.2 — Haptic Sleeve Pairing & Calibration

What: Bluetooth pairing for up to four sleeves (left arm, right arm, left leg, right leg). A short calibration test confirms each sleeve's vibration motor is working and that the user can distinguish zones. The resulting `sleeve_sensitivity_map` determines which body parts will receive haptic feedback vs. spatial audio routing in Phase 3.

How: Web Bluetooth API (Chrome + Android). Each sleeve is identified by UUID. A calibration sequence fires each zone in turn; user confirms via voice ("Yes, I felt that") or keypress.

Method: `[STAT]` Hardware handshake, no model.

PHASE 1 — Assistance Preparation Pipeline

Triggered when a user submits a YouTube URL. Runs asynchronously on the Python backend. User is notified when the sidecar manifest is ready. The original YouTube video is never downloaded for playback — only audio/frames are fetched transiently for analysis, then discarded. See Open Questions on terms-of-service constraints.

F1.1 — YouTube Video Import & Metadata Extraction

What: User pastes a YouTube URL. The system fetches publicly available video metadata via the YouTube Data API v3: video ID, title, channel name, duration, thumbnail URL, available caption tracks, and published date. The video is registered in the user's library with this metadata. No video file is stored.

For sidecar analysis (F1.2–F1.8), audio and sampled frames are fetched transiently using yt-dlp, processed in memory, and discarded. Processed artifacts stored are metadata and the sidecar manifest only.

How: YouTube Data API v3 for metadata. Python backend with yt-dlp for transient audio/frame extraction during preprocessing. Status updates via Server-Sent Events to Next.js frontend.

Method: `[STAT]` No model for metadata fetch.

F1.2 — Exercise Timeline Anchoring

What: The video is analyzed to identify discrete exercise segments and map them to timestamps. For each segment the sidecar manifest records: start/end timestamps, exercise name, primary and secondary body parts, and a brief natural-language description of the movement (used to generate haptic/spatial cues and orient form correction, **not** to narrate over the trainer).

How: Gemini 1.5 Pro with video input (sampled frames + audio transcript). Structured JSON schema enforced in output. Few-shot examples anchor the description style to AudioToDes D1

format (name → beat → movement → body part → direction → position) for internal use.

Prompt engineering strategy: System prompt specifies that descriptions are for internal accessibility calibration, not user narration. Output includes `(description_internal)` (D1 format, for cue generation) and `(description_accessible)` (≤15 words, for use by the assistant only when the user asks "what is this exercise?"). Chain-of-thought instructed: "First identify the exercise name, then identify which body part moves most."

Future upgrade path: Once 500+ video segments are labeled, SFT on Gemini Flash to replace few-shot prompting. `[SFT candidate]`

Method: `[PE]` — Gemini 1.5 Pro, structured few-shot prompting.

F1.3 — Trainer Instruction & Caption Timestamp Extraction

What: The video's audio transcript (via Whisper) and available caption track are analyzed to extract timestamps of trainer instructions: exercise cues ("Now squat down"), rep counts ("One, two, three..."), form reminders ("Keep your back straight"), rest cues, and transition announcements. These timestamps are stored in the sidecar manifest as `(trainer_instruction_events[])`.

Why critical: This is the foundation of the creator-first policy. The assistant uses these timestamps to time its own output into natural gaps. It also prevents FitA11y from speaking over the trainer's form reminders (which would be redundant and disorienting).

How: Whisper transcription on the audio track. Gemini Flash analyzes the transcript and classifies each utterance by type: `(REP_COUNT)`, `(FORM_CUE)`, `(EXERCISE_TRANSITION)`, `(ENCOURAGEMENT)`, `(REST_CUE)`, `(DEMONSTRATION_ONLY)`, `(SILENCE)`. `(SILENCE)` windows are tagged as `(assistant_may_speak: true)`.

Method: `(STAT)` (Whisper transcription) + `(PE)` (Gemini Flash classification).

F1.4 — Expected Movement Windows & Rep Thresholds

What: For each exercise segment, MediaPipe Pose is run on sampled frames (every 5th frame) to compute joint angles at all 33 landmarks. The counting joint (joint with maximum angle range), minimum angle (bottom of rep), maximum angle (top of rep), and expected rep duration (in seconds) are stored in the sidecar manifest. These values define the expected movement window for real-time comparison during live assisted playback.

How: Python + MediaPipe Pose on the backend. For each joint: `(range = max_angle_across_frames - min_angle_across_frames)`. Joint with max range = counting joint. Tolerance band: ±15° default (adjusted per session by F2.4).

Method: `[LOCAL]` / `[STAT]` — pure pose math, no model.

F1.5 — LLM Evaluator (Counting Joint Sanity Check)

What: After MediaPipe identifies the counting joint, an LLM sanity-checks whether it makes anatomical sense for the named exercise. If implausible (e.g., wrist identified as counting joint for a squat due to arm swing), the evaluator rejects it and suggests the correct joint.

How: Gemini Flash. Input: exercise name + identified joint + angle range. Output: `{valid: bool, suggested_joint: string, reason: string}`. If invalid, retries F1.4 with the suggested joint forced.

Prompt: `"For a [exercise], is [joint] with range [min°-max°] a sensible counting joint? Reply in JSON."`

Method: `[PE]` — Gemini Flash, single-turn structured prompt.

F1.6 — Likely Form Risks & Correction Templates

What: For each exercise, the system pre-generates a library of 8-12 correction phrases covering the most common form errors (e.g., for a squat: "Knees tracking inward — push them out," "Hips not low enough — sit deeper"). These are stored in the sidecar manifest and served instantly during live assisted playback without an API call.

This replaces v1's F1.7. The framing shifts: these are form risk annotations on the existing exercise, not a replacement exercise script.

Why important: Covers 80%+ of real-time correction needs without API latency. The LLM is only invoked for novel or compound errors not in the template set.

How: Gemini 1.5 Pro. Prompt: `"List the 10 most common form errors for [exercise] and a brief correction phrase (≤8 words) for each: body part + direction + position."`
Output stored as `form_risk_templates[]` in the manifest.

Method: `[PE]` — Gemini 1.5 Pro, list-format prompt with AudioToDes D1 schema.

F1.7 — Haptic & Spatial Cue Profile Generation

What: For each exercise, a haptic cue profile is generated: which sleeve(s) activate, at which moment in the rep cycle (start/peak/end), what pattern (pulse / sustained / wave), and at what intensity (1-5). For body parts without sleeve coverage, the profile specifies a spatial audio cue coordinate instead (see F3.9). Stored in the sidecar manifest.

How: Two-tier system.

- **Tier 1 (Template library):** ~30 common exercises have hand-authored haptic profiles stored in the database. Used directly if exercise name matches.
- **Tier 2 (LLM-generated):** For unrecognized exercises, Gemini Flash generates a profile.
Prompt: `"For [exercise] targeting [body parts], suggest haptic cues for a BLV user with sleeves on both arms and both legs. Return JSON: {sleeves[], timing, pattern, intensity, spatial_audio_fallback[]}."` Spatial audio fallback coordinates are included for any body region without sleeve coverage.

Method: Tier 1: `[STAT]`. Tier 2: `[PE]` — Gemini Flash.

F1.8 — Beat & Rhythm Extraction

What: The video's audio track is analyzed to detect the underlying rhythm (BPM) or explicit beat cues (the trainer counting "1, 2, 3, 4"). Beat timestamps are stored in the sidecar manifest so the pacing layer (F3.6) can align haptic rhythm cues to the video's native beat — reinforcing the trainer's own count rather than replacing it.

How: Python librosa for BPM detection. Gemini Flash analyzes the Whisper transcript to extract explicit count cues where the trainer verbalizes them.

Method: `[STAT]` (librosa BPM) + `[PE]` (beat-cue extraction from transcript).

F1.9 — Silence & Speaking Opportunity Map

What: Combines the outputs of F1.3 and F1.8 to produce a `speaking_opportunity_map[]` — a timeline of timestamps where the assistant is permitted to speak, the available window duration, and the recommended delivery mode (haptic-only / gap-timed / duck-and-speak). Windows shorter than 1.5 seconds are marked `haptic_only: true`.

How: `[STAT]` post-processing of `trainer_instruction_events[]` and beat timestamps. No model call.

Output fields per window: `{start_ms, end_ms, duration_ms, mode: "haptic_only" | "gap_timed" | "duck_speak", context: "between_reps" | "rest_period" | "transition" | "silence"}`

Method: `[STAT]`

PHASE 2 — Pre-Session Setup

Runs each time a user starts a workout. Takes 1-2 minutes.

F2.1 — Video Library & Workout Selection

What: The user's library of processed YouTube videos. Each entry announces (via TTS): video title, creator/channel name, duration, exercise list, and last-session summary if previously attempted. Screen-reader fully compatible: all information in text and ARIA labels, no visual-only elements.

BLV-specific details:

- Voice navigation: user can say "Next video," "Play [video title]," "Describe this workout."
- Thumbnail descriptions are auto-generated during preprocessing (Gemini Flash: "Describe this thumbnail in 10 words for a BLV user.").

Method: `[STAT]` UI feature. `[PE]` for thumbnail descriptions.

F2.2 — Coach Persona Selection

What: The user selects an AI assistant personality for supplementary cues. Three archetypes: **Hype** (high-energy, brief), **Technical** (precise, anatomical), **Supportive** (gentle, patient). The chosen persona governs the style of FitA11y's interventions only — it does not affect the trainer's voice or narration.

Persona prompts:

- **Hype:** "You are an energetic sideline coach. Your cues are ≤6 words. Fire up, brief, supportive. Never talk over the trainer."
- **Technical:** "You are a precise physiotherapist. Use anatomical terms. Explain why. Calm, brief. Never contradict the trainer."
- **Supportive:** "You are a patient, warm coach. Frame corrections as invitations. ≤10 words. Never use imperative voice."

Why not training: Style variation, not capability variation. Persona is prompt-driven and A/B-testable without retraining.

Method: `[PE]` — System prompt injection.

F2.3 — Audio & Voice Customization

What: TTS speed (0.75×–1.5×), voice gender/style, correction frequency, interruption level,

verbosity setting, and spatial audio toggle. All coexistence settings (see Audio Coexistence Rules) are configurable here.

How: Settings stored in user profile. Passed to Google Cloud TTS for audio rendering and injected into system prompts as constraints (e.g., `"User verbosity setting: SHORT. Keep all responses under 6 words."`).

Method: `[STAT]` Settings → prompt parameters.

F2.4 — Difficulty & Sensitivity Calibration

What: Short pre-session questionnaire: "First time with this workout?" / "How are you feeling today? (Fresh / Normal / Tired)." Answers adjust the form correction sensitivity threshold.

- Fresh: $\pm 10^\circ$
- Normal: $\pm 15^\circ$
- Tired: $\pm 20^\circ$

Historical `error_trend` from the user model (F5.2) can override the questionnaire result.

Method: `[STAT]` Rule-based threshold adjustment.

F2.5 — Camera & Sleeve Connection Check

What: MediaPipe is initialized in-browser. The system confirms pose detection is functional. All connected sleeves fire a brief test pulse. A voice prompt guides the user to stand in the starting position described in the sidecar manifest for the first exercise.

How: MediaPipe Tasks Vision (WebAssembly, in-browser). Web Bluetooth sleeve status check. Starting position prompt served from the sidecar manifest's `description_accessible` field.

Method: `[LOCAL]` + `[STAT]`

F2.6 — Pre-Workout Exercise Q&A

What: Before starting, the user can ask questions about any exercise in today's workout: "What should I feel in my glutes during a squat?" / "Where do I put my hands for a shoulder press?" FitA11y answers using the sidecar manifest data plus Gemini's general knowledge.

Why novel: No prior BLV fitness system offers exercise pre-briefing. This reduces mid-session confusion and supports users with no prior exposure to an exercise — without requiring them to pause the video mid-workout.

Method: `[PE]` — Gemini Flash, exercise context in prompt.

PHASE 3 — Live Assisted Playback

The core real-time loop. The embedded YouTube IFrame player is the central UI element. FitA11y synchronizes its assistance layer to the player's current timestamp. The creator's audio and video are never suppressed, regenerated, or replaced.

F3.0 — Embedded YouTube Player & Playback Controls

What: The session UI centers on an embedded YouTube IFrame player. FitA11y wraps it with a BLV-accessible control layer.

BLV voice commands (Web Speech API):

- "Pause" / "Play"
- "Slow down" → sets playback to 0.75×
- "Rewind 10 seconds"
- "Repeat last instruction" → rewinds to the most recent `trainer_instruction_event` timestamp and replays
- "Ask assistant" → pauses video and opens Q&A
- "Continue" → resumes video
- "Mute assistant for this set" → suppresses FitA11y output until next exercise transition
- "Skip to [exercise name]" → seeks to the corresponding exercise anchor timestamp

How: YouTube IFrame API for playback control. Web Speech API for voice commands. All controls also available as large-target accessible buttons with ARIA labels.

Method: `[STAT]` + `[LOCAL]`

F3.1 — Live Pose Estimation (MediaPipe)

What: Device camera feeds into MediaPipe Pose running in-browser at up to 30fps. At every frame, the system computes the current angle of the counting joint and secondary joints. This data feeds all real-time assistance agents. No frames are sent to the server — only derived events (rep completed, error detected, angle value).

Latency: <16ms per frame on modern mobile hardware.

Method: [LOCAL] — MediaPipe, no API call.

F3.2 — Rep Counting with Haptic Confirmation

What: Supplementary rep confirmation for BLV users. When the counting joint completes a rep (angle crosses top threshold, then returns below bottom threshold): a short single-pulse haptic fires on the primary limb sleeve, and the TTS speaks the rep number ("Eight.") **only if** the trainer is not currently speaking and the verbosity setting permits numeric cues.

This is a supplementary layer. If the trainer is counting aloud, the assistant defaults to haptic-only confirmation unless the user has enabled "always announce reps."

How: [LOCAL] threshold crossing detection with hysteresis (joint must hold above top threshold for ≥ 2 frames). Haptic via Web Bluetooth. TTS via browser SpeechSynthesis (no network call for single-word output). Trainer speech detection via real-time amplitude monitoring of the YouTube player audio track.

Method: [LOCAL] / [STAT]

F3.3 — Real-Time Form Correction (Correction Assistant)

What: When a joint deviates outside its acceptable range for ≥ 3 consecutive frames, a correction is triggered. Delivery follows the audio coexistence rules: haptic-first, speech only in an available gap or on duck-and-speak setting. Corrections follow AudioToDes D2 format: body part + direction + position.

Three-tier dispatch:

- **Tier 1 (Pre-generated template, <50ms):** Error type matches a template from F1.6 → served immediately. Delivery mode determined by the speaking_opportunity_map (F1.9). If no gap is available: haptic-only.
- **Tier 2 (Gemini Flash, ~350ms):** Novel or compound error → Gemini Flash generates a correction. Input: exercise name, joint, current angle, acceptable range, persona, last 3 corrections (to avoid repetition), current speaking_opportunity_map window.
- **Tier 3 (Suppression):** Same error triggered 4+ consecutive times without improvement → stop correcting and escalate to pacing assistant (F3.5).

Prompt (Tier 2): "[Persona prefix]. User is doing [exercise]. Their [joint] is at [angle]°, should be in [range]°. Give ONE correction in ≤ 8 words: body part +

direction + position. Do not repeat: [last_3_corrections]. Trainer audio is [active/paused]."

Method: Tier 1: [STAT], Tier 2: [PE] — Gemini Flash.

F3.4 — Adaptive Haptic Feedback Selection

What: Per user per exercise, the system learns which correction delivery mode is most effective: audio-only, haptic-only, simultaneous haptic+audio, or suppressed (trust user to self-correct). A contextual bandit model selects the mode for each correction event and updates based on whether the error recurred on the next rep.

Why novel: No prior system adapts correction modality. First application of bandit-based feedback selection in BLV exercise coaching.

How: Thompson Sampling contextual bandit. Context vector: [exercise_id, error_type, rep_number, session_fatigue_index, user_prior_accuracy]. Reward: +1 if error did not recur on next rep, -1 if it did, 0 if exercise changed. State persists across sessions in the database. Lightweight statistical model (~100 lines Python), not a neural network.

Method: [BANDIT] — Thompson Sampling, no LLM.

F3.5 — Adaptive Pacing Assistance

What: FitA11y monitors whether the user is keeping up with the video's pace over a rolling 3-rep window. If the user is lagging, FitA11y offers playback adaptations. **The canonical workout (the creator's video) is never rewritten.** Adaptations are delivery changes and user-facing recommendations only.

Lag detection: $\text{lag_ratio} = \text{user_rep_duration} / \text{expected_rep_duration}$. If $\text{lag_ratio} > 1.2$ for 2 consecutive reps, an intervention is triggered.

Four escalating interventions:

1. **Slow prompt (lag $\leq$ 20% behind):** FitA11y says (in a gap or via haptic) "Take your time" and sets playback to $0.75\times$ via YouTube IFrame API.
2. **Hold cue (lag 21–40% behind):** Video pauses. FitA11y says: "Pausing — take this rep at your pace." Sustained low haptic on primary sleeves. Video resumes when the user completes the rep.
3. **Adaptation recommendation (lag $>$ 40% behind, 3 consecutive reps):** System pauses video and says: "You might want to do fewer reps of this one — up to you. Say 'keep going' to continue as-is, or 'reduce reps' to track fewer." If the user says "reduce reps," FitA11y adjusts its own rep-tracking target only — the video continues unchanged.

4. **Modification offer (lag > 40% behind AND form errors increasing):** FitA11y pauses the video and says: "The trainer shows a harder version here. Want a lower-impact alternative? Say 'yes' for a suggestion or 'continue' to keep going."

Recovery: When $\text{lag_ratio} < 1.1$ for 2 reps, playback gradually returns to 1× and FitA11y says "Great, back to pace."

User override: "Keep going" at any time dismisses the intervention and continues at original pace.

Method: Lag detection: [STAT] . Intervention speech: [PE] — Gemini Flash with session context.

F3.6 — Beat & Rhythm Reinforcement

What: Between correction events, FitA11y reinforces the video's own beat via haptic pulses timed to the extracted beat (F1.8). This helps BLV users pace their reps to the trainer's rhythm nonvisually. When the user's pace drifts by more than $\pm 20\%$ from the video's beat, FitA11y adds a brief nudge: "Slower — two beats" or "Pick it up." This handles rhythm drift (too fast, or irregular), distinct from F3.5 (systematically lagging).

Haptic default: Beat reinforcement is haptic-only during trainer speech. A short verbal nudge is permitted only in a gap window.

Method: [STAT] (beat comparison + haptic timing) + [PE] (speech nudge, Gemini Flash).

F3.7 — Motivational Cues (Milestone Events)

What: Positive reinforcement triggered by milestone events: rep PR for this exercise, completing a set, reaching the workout halfway point, recovering from a flagged form error. FitA11y delivers a short cue in the user's chosen persona.

Motivational cues are low-priority (see Audio Coexistence Rules). During trainer speech, they default to haptic-only (a distinct celebratory pattern: three quick pulses). Spoken cues are reserved for rest periods or natural gaps.

How: Events detected by [STAT] rules. Speech generated by Gemini Flash (≤ 12 words) with milestone and persona as context. Prompt: $\text{[Persona prefix]. User just [milestone event]. ONE encouraging phrase } \leq 10 \text{ words.}"$

Method: [STAT] (event detection) + [PE] (speech, Gemini Flash).

F3.8 — Interactive Mid-Session Q&A NEW

What: At any point during the workout, the user can ask FitA11y a question by voice or button press. Video pauses before FitA11y responds; trainer audio resumes after.

Example questions:

- "My shoulder hurts on the press — should I stop?"
- "What muscle is this working?"
- "How many reps has the trainer said to do?"
- "Make this easier"
- "Repeat the last thing the trainer said"

How: Web Speech API for voice input. Gemini Flash (or Gemini Pro for complex questions) with full session context: current timestamp, current exercise, rep count, error history, `trainer_instruction_events[]` near current timestamp. TTS response.

Safety rule in the prompt: "If the user reports pain or injury, always advise them to stop the exercise and rest. Never encourage exercising through pain. This is non-negotiable."

Note on "what did the trainer say": For questions about the trainer's instructions, FitA11y consults the sidecar manifest's caption/transcript data and reports what was said, rather than paraphrasing or interpreting it.

Method: `[PE]` — Gemini Flash/Pro with session context.

F3.9 — Spatial Audio for Uncovered Body Parts NEW

What: For body parts without sleeve coverage (back, core, neck, hips on certain configurations), form corrections are conveyed via directional spatial audio rather than haptics. Audio sounds like it originates from the relevant body region (e.g., a forward-head-posture correction sounds from behind/above; a core correction sounds central).

Why novel: Prior work uses spatial audio for orientation (PunchPulse). This is the first use of spatial audio for body-part-specific real-time form correction in BLV fitness.

How: Web Audio API PannerNode with HRTF model. Each body region maps to a 3D coordinate. The system detects which body parts have active sleeves from `sleeve_sensitivity_map` and routes corrections accordingly: haptic if sleeve present, spatial audio if not.

Method: `[LOCAL]` — Web Audio API, no model.

F3.10 — Fatigue-Aware Feedback Scaling

What: In the final 30% of a set (e.g., reps 8-10 of a 10-rep set), FitA11y automatically reduces feedback intensity: the angle tolerance band widens by 5°, and correction language shifts from imperative ("Fix your knees") to supportive ("Almost there — keep it controlled"). This prevents over-coaching at the point where fatigue is expected and the trainer themselves would typically normalize effort.

Why: AudioToDes found participants were frustrated by feedback that did not account for physical limits. This directly implements that finding.

How: `[STAT]` rep counter triggers fatigue mode. `[PE]` persona prompt appends: `"User is on their final reps – use encouraging tone, not corrective imperatives."`

Method: `[STAT]` + `[PE]` — prompt modifier.

F3.11 — Accessible Visual Description (Visual-Only Demonstrations)

What: When the trainer's segment includes a visual-only demonstration (no spoken instruction, movement shown silently), FitA11y detects the silence window and provides an audio description of the movement visible in the video, using the sidecar manifest's `description_accessible` field and the current frame analysis.

Why needed: BLV users cannot benefit from visual-only form demos. This is the one case where FitA11y's narration fills a genuine gap rather than supplementing existing trainer audio.

How: Silence windows flagged `context: "demonstration_only"` in the `speaking_opportunity_map` (F1.9) trigger this feature. Description served from the pre-generated `description_accessible` field. If the demo is not in the manifest, Gemini Flash generates a brief description from the frame and transcript context.

Method: `[STAT]` (silence window detection) + `[PE]` (Gemini Flash for novel demos).

PHASE 4 — Post-Session Summary

F4.1 — Session Summary Report

What: After the workout ends, FitA11y generates a spoken and visual summary covering: exercises completed, reps tracked per set, form accuracy per exercise (% of reps with no flagged errors), most common error types, pace consistency score, and comparison to the previous session. The summary clearly distinguishes between the trainer's prescribed workout and the user's tracked performance.

How: Session data aggregated from the database. Summary text generated by Gemini Flash. Spoken via TTS. Visual version (for sighted companions) displayed in the app with accessible ARIA equivalents.

Method: `[PE]` — Gemini Flash with session statistics in prompt.

F4.2 — User Model Update

What: After each session, the user model is updated with new performance data: form accuracy trends per exercise, average rep duration, error type frequency, which correction modality worked best (from F3.4 bandit), and which gap windows were most effective for assistant speech.

How: Python backend aggregates session data. Updates `users.per_exercise` JSON. No model training — statistical bookkeeping. Updated user model injected into system prompts for the next session.

Method: `[STAT]` — Database update, no model call.

F4.3 — Progress History & Trends

What: Browsable session history showing improvement over time: rep count trends, form accuracy trends per exercise, fatigue pattern analysis. All data presented in audio-accessible format — charts have spoken equivalents announced via ARIA live regions. The history view also shows each video watched, linking back to the original YouTube URL.

How: Next.js pages reading from `sessions` and `feedback_events` tables. Accessible color palettes; all chart values announced via ARIA.

Method: `[STAT]` UI feature.

Training & Prompting Strategy Summary

Component	Technique	Status
Exercise timeline anchoring	<code>[PE]</code> structured prompting, AudioToDes D1 schema	Current
Trainer instruction classification	<code>[PE]</code> Gemini Flash utterance classifier	Current
Correction template generation	<code>[PE]</code> list-format prompting	Current
Haptic cue generation	<code>[PE]</code> structured JSON prompting	Current

Component	Technique	Status
LLM counting-joint evaluator	<code>[PE]</code> binary structured prompt	Current
Beat/rhythm extraction	<code>[STAT]</code> librosa + <code>[PE]</code> beat-cue extraction	Current
Silence/gap map generation	<code>[STAT]</code> post-processing	Current
Persona voices	<code>[PE]</code> system prompt injection	Current
Adaptive modality selection	<code>[BANDIT]</code> Thompson Sampling	Current
Adaptive pacing intervention	<code>[STAT]</code> lag ratio + <code>[PE]</code> speech	Current
User capability model	<code>[STAT]</code> session statistics	Current
Exercise description fine-tuning	<code>[SFT]</code> Gemini Flash on curated D1 dataset	Future work
Persona preference fine-tuning	DPO on BLV user study preference pairs	Future work

Where SFT Would Be Applied (Future)

Description generation fine-tuning (highest ROI):

- Dataset: 500–1,000 exercise video segments labeled with D1-format `description_accessible` fields
- Base model: Gemini Flash
- Platform: Google Vertex AI
- Effect: Eliminates elaborate few-shot prompting; model learns the D1 schema as a prior

DPO for persona preference:

- Collect pairs of assistant utterances from BLV user studies; users rate preference
- Apply DPO per persona archetype
- Produces genuinely personalized assistant voices that adapt to individual communication preferences

Why Not PPO/GRPO

PPO and GRPO require a well-defined reward function at inference time. In live exercise assistance, the reward (did the user's form actually improve?) is delayed, noisy, and confounded by fatigue. The bandit approach (F3.4) converges faster with sparse data and is far simpler to implement and explain in a paper.

Open Questions & Product Decisions

The following questions require explicit product and legal decisions before implementation begins.

1. YouTube download vs. embed

Question: Should preprocessing use yt-dlp to transiently download audio/frames for sidecar analysis, or should the app rely exclusively on the YouTube IFrame API and caption tracks for all analysis?

Tradeoffs:

- yt-dlp gives higher-quality audio/frame access for pose analysis and transcription but creates YouTube Terms of Service exposure. Transient processing with no storage may be acceptable under fair use arguments, but this requires legal review.
- IFrame-only approach is ToS-safe but caption track availability is inconsistent (auto-generated captions vary significantly in accuracy), and frame access for MediaPipe preprocessing is unavailable.

Recommendation: Proceed with yt-dlp for transient preprocessing only. Obtain legal review. Store nothing beyond the sidecar manifest. Add a ToS acknowledgment in onboarding.

2. Caption availability

Question: How should FitA11y handle videos without captions or with low-quality auto-generated captions?

Options:

- Always run Whisper transcription regardless of caption track availability (consistent quality, higher compute cost)
- Use available captions when quality exceeds a confidence threshold, fall back to Whisper
- Flag low-caption-quality videos in the library and warn users before processing

Recommendation: Always run Whisper as a parallel track and merge with available captions. Flag videos where transcript confidence is below threshold with a warning: "Some trainer instructions may not be accurately detected for this video."

3. Copyright and creator consent

Question: Is there a creator consent or attribution requirement before a video can be added to

Considerations:

- FitA11y does not reproduce the video, redistribute it, or generate replacement content. The original video plays via the YouTube IFrame embed (creator's ad revenue is preserved).
- The sidecar manifest is a derivative annotation work — likely fair use, but not settled law.
- Some creators may object to AI systems annotating their content.
- YouTube's API Terms of Service require that content be displayed within an IFrame embed and not downloaded for storage.

Recommendation: Add a creator-opt-out mechanism (a URL or channel-level blocklist).

Consider a public statement of the creator-first policy to preempt objections. Do not process unlisted or private videos.

4. Intervention aggression level

Question: How proactively should FitA11y intervene? Should it err toward more corrections (higher recall, more interruptions) or fewer (lower recall, less disruption)?

Considerations:

- BLV users have higher cost from missed form errors (safety) but also higher cost from excessive interruptions (breaks immersion, contradicts trainer).
- AudioToDes findings: users preferred brief, infrequent corrections over frequent, detailed ones.
- F3.4 bandit will learn this per-user over time, but cold-start behavior matters.

Recommendation: Default to conservative (haptic-first, speech in gaps only, corrections throttled to one per rep per joint). Provide an explicit "More guidance" toggle for new users who want higher verbosity during their first sessions.

5. Trainer audio ducking legality

Question: Is dynamically ducking the original YouTube audio track to speak over it compliant with YouTube's terms?

Note: Audio ducking is applied only to the local playback stream in the user's browser — the original audio is not modified, re-encoded, or redistributed. This is analogous to a hearing aid applying audio processing.

Recommendation: Likely permissible as a local accessibility accommodation. Confirm with legal. Document in privacy policy that no audio modification is stored or transmitted.

6. Offline / low-connectivity support

Question: Should any part of the sidecar manifest or assistance pipeline work offline?

Consideration: YouTube requires network connectivity. However, the sidecar manifest (once generated) could be cached locally to allow offline review of correction templates, exercise descriptions, and haptic profiles. Live pose estimation runs locally already.

Recommendation: Cache the sidecar manifest on-device after first processing. Offline mode supports Phase 0-2 features and pre-workout Q&A. Phase 3 (live playback) requires connectivity.

End of Feature Specification v2.0